

Analyse critique — Activité 10

Projet fil rouge : RPG 40K Survivor **Module :** Coordination Front & Back — M2 INFO (Ynov Connect)

Ce document constitue le rapport d'analyse critique demandé en activité 10 : points forts / faibles, pistes de refactoring, apports du module, feuille de route post-module et compte-rendu du plan d'action mis en œuvre.

1. Points forts et points faibles

Points forts

- **Séparation claire des couches** : présentation (React), API (FastAPI), domaine métier (`src/`). Le code métier est indépendant du web.
- **Communication front/back maîtrisée** : REST pour les actions, **SSE** pour la narration en flux, ce qui offre une vraie expérience temps réel.
- **Sécurité conforme aux attendus** : JWT signés, mots de passe hachés bcrypt, gestion de rôles `player` / `admin`, routes protégées (401/403).
- **Fonctionnalité IA réelle** : narration générée par OpenAI, avec **repli local** déterministe garantissant une démonstration fiable même sans réseau.
- **Persistance robuste** : comptes en SQLite, sauvegardes isolées par utilisateur.
- **Tests présents** : suite backend (dont l'authentification) et tests frontend.

Points faibles

- **Sessions de jeu en mémoire** : l'état de partie vit dans des dictionnaires côté serveur ; un redémarrage recharge depuis YAML mais perd la session live.
- **Complexité cognitive élevée** de certaines fonctions (`combat_action`, `streamSSE`) signalée par l'analyse statique.
- **CORS permissif** (`allow_origins=["*"]`) adapté au dev mais à restreindre en prod.

- **Rafraîchissement du jeton** absent : pas de refresh token, seulement un access token à durée de vie longue.
- **Couplage sauvegarde YAML / logique** : la persistance de partie n'est pas en base.

2. Parties à refactoriser

Cible	Problème	Refactoring proposé
<code>combat_action</code>	Complexité 27 > 15	Extraire <code>_resolve_player_action</code> , <code>_resolve_enemy_turn</code>
<code>streamSSE</code> (front)	Complexité 25 > 15	Extraire le parsing de ligne SSE dans une fonction dédiée
Persistance partie	YAML dispersé	Centraliser dans un dépôt/ <code>Repository</code> unique
CORS	Trop ouvert	Lister explicitement les origines autorisées
Constantes SQL	Littéraux dupliqués	Extraire les requêtes récurrentes en constantes

3. Apports du module pour le projet

Le module « Coordination Front & Back » a directement structuré le projet :

- mise en place d'une **API REST propre** (verbes/codes HTTP, gestion d'erreurs) ;
- **sécurisation JWT** avec hachage et rôles (activité 5) ;
- articulation **frontend** ↔ **backend** avec gestion des états et des flux de données ;
- réflexe de **documentation technique** et de **modélisation des données** (MCD/MLD) ;
- recul par l'**analyse critique** et la priorisation des risques.

4. Feuille de route post-module

Risques identifiés

Risque	Type	Gravité
Perte de session live au redémarrage backend	Technique	Moyenne
Secret JWT par défaut si non configuré	Sécurité	Élevée
CORS ouvert en production	Sécurité	Moyenne
Absence de refresh token	UX / Sécurité	Faible
Complexité de fonctions clés	Maintenabilité	Moyenne

Plan d'action priorisé

Priorité	Action	Statut
P0	Exiger un <code>JWT_SECRET</code> fort en prod (échec si valeur par défaut)	À planifier
P0	Authentification JWT + hachage + rôles	✅ Fait pendant le module
P1	Restreindre le CORS aux origines connues	À planifier
P1	Refactorer <code>combat_action</code> et <code>streamSSE</code>	À planifier
P2	Persister les sauvegardes en base plutôt qu'en YAML	Backlog
P2	Ajouter un refresh token	Backlog

5. Mise en œuvre du plan d'action (réalisé)

Les tâches **critiques pour le rendu** ont été traitées durant le module :

1. **Sécurisation complète (P0)** — implémentation de l'authentification JWT, du hachage bcrypt et des rôles, avec routes protégées et tests dédiés (`backend/auth.py`, `tests/test_api.py`).
2. **Frontend d'authentification** — écran connexion/inscription, stockage du jeton, envoi automatique de l'en-tête `Authorization`, déconnexion (`frontend/src/components/AuthPanel.jsx`, `frontend/src/api.js`).

- 3. **Documentation de rendu** — document de cadrage, MCD/MLD, wireframes, doc technique et cette analyse critique ([docs/module/](#)).
- 4. **Validation** — suite de tests backend au vert (21 tests) et build frontend OK.

6. Analyse SWOT

Forces (internes +)	Faiblesses (internes -)
Architecture en couches nette (présentation / API / domaine)	État de jeu live en mémoire, non persisté à chaud
Sécurité JWT + bcrypt + rôles opérationnelle	CORS permissif, secret JWT par défaut si non configuré
IA réelle avec repli local fiable	Dépendance à un service externe payant (OpenAI)
Suite de tests back + front, build reproductible	Complexité élevée de <code>combat_action</code> et <code>streamSSE</code>
Déploiement conteneurisé fonctionnel (VPS)	Sauvegardes YAML hors base relationnelle
Opportunités (externes +)	Menaces (externes -)
Migration facile vers PostgreSQL/refresh token	Coût / facturation OpenAI (<code>billing_not_active</code>)
Extension multijoueur, nouveaux contenus narratifs	Évolution/rupture d'API du modèle IA
Portfolio valorisable (démon public en ligne)	Exposition de secrets si mauvaise gestion <code>.env</code>

7. Indicateurs qualité (métriques)

Indicateur	Valeur	Commentaire
Tests backend	21 tests au vert	Couvre auth, routes protégées, hachage
Tests frontend	13 tests au vert	Couvre tokens, appels API

Indicateur	Valeur	Commentaire
Build frontend	OK	Aucune erreur bloquante
Routes API exposées	17	REST + SSE, documentées Swagger
Modules domaine (<i>src/</i>)	11	Découplés du framework web
Couches applicatives	3	Présentation / API / domaine
Fonctions à complexité > seuil	2	<i>combat_action</i> , <i>streamSSE</i> (refactoring priorisé)
Disponibilité démo	Repli local garanti	Aucune page blanche même si IA indisponible

Ces indicateurs servent de **base de référence** : ils permettront de mesurer objectivement l'effet des refactorings et durcissements planifiés en section 4.

8. Conclusion

Le projet atteint les objectifs du module : jeu fullstack jouable, communication front/back en REST + SSE, **sécurité JWT opérationnelle**, IA réellement intégrée et persistance. Les axes d'amélioration sont identifiés, priorisés et documentés, ce qui constitue une base saine pour la suite (refactoring, durcissement production).